

# Credit Card Default Classification

BITS WILP M.Tech (AIML) — Machine Learning Assignment 2. Six classifiers trained on one shared preprocessing pipeline, compared on six metrics, and served through a Streamlit app.

**Live app:** <https://credit-card-classification.streamlit.app/> **Repo:** <https://github.com/shashisingh0015/credit-card-classification>

## a. Problem statement

Predict whether a credit card client will default on their payment **next month**, from their credit limit, demographics, and six months of billing and repayment history. This is a **binary classification** problem: a bank deciding credit limits, collections priority, or early-intervention outreach needs a risk score per client, not just an aggregate default rate.

The dataset's class imbalance (roughly 4 non-defaulters for every defaulter) makes this a genuinely interesting classification problem rather than a solved one — a model that never predicts default already scores 77.87% accuracy while being completely useless, which is why this project treats **MCC and AUC-ROC as the metrics that matter**, not accuracy (see section d).

## b. Dataset description

UCI "Default of Credit Card Clients" (Taiwan, 2005), UCI repository id 350 ([archive.ics.uci.edu/dataset/350](https://archive.ics.uci.edu/dataset/350)).

|                |                                                                            |
|----------------|----------------------------------------------------------------------------|
| Instances      | 30,000 raw → <b>29,965</b> after dropping 35 exact duplicate rows          |
| Features       | 23                                                                         |
| Target         | default_next_month — binary (0 = no default, 1 = default)                  |
| Class balance  | 77.88% no default / 22.12% default                                         |
| Missing values | none                                                                       |
| Split          | stratified 80/20 → 23,972 train / 5,993 test, positive rate 22.13% in both |

The UCI source serves columns as opaque `X1..X23` ; this project renames them on load to their documented meaning:

| Raw | Feature                         | Raw    | Feature                                         |
|-----|---------------------------------|--------|-------------------------------------------------|
| X1  | LIMIT_BAL — credit limit (NT\$) | X12–17 | BILL_AMT1..6 — monthly bill amount              |
| X2  | SEX                             | X18–23 | PAY_AMT1..6 — monthly amount paid               |
| X3  | EDUCATION                       | X6     | PAY_0 — most recent repayment status            |
| X4  | MARRIAGE                        | X7–11  | PAY_2..PAY_6 — repayment status, 5 prior months |
| X5  | AGE                             | Y      | default_next_month (target)                     |

Two data quirks were investigated during EDA ( `model/eda.ipynb` ) and handled explicitly rather than ignored:

- **EDUCATION and MARRIAGE contain undocumented codes** ( `EDUCATION`  $\in \{0, 5, 6\}$ , `MARRIAGE` = 0) covering ~1.3% of rows between them. Both are folded into each column's documented "others" bucket rather than dropped.
- **PAY\_0 's risk is not monotonic in its raw code.** UCI documents only -1 (paid duly) and 1-9 (months late), but the data also contains -2 (no credit used) and 0 (revolving credit) — and 0 is the single most common value, at ~49% of rows. Measured default rate by code: -2 → 13.2%, -1 → 17.0%, 0 → 12.6%, 1 → 34.2%, 2 → 69.3%. The real structure is a **threshold** at `PAY_0`  $\geq 1$  (13.8% vs 50.5% default rate, a 3.7× ratio), which the six months of `PAY_*` columns turn out to be the six strongest predictors in the dataset. This single finding predicted, correctly, that Logistic Regression would trail every tree-based model (section e) — a linear coefficient cannot express a non-monotonic risk curve.

Every model's preprocessing ( `model/preprocess.py` ) standard-scales the 14 numeric + 6 ordinal `PAY_*` columns and one-hot encodes the 3 nominal columns ( `SEX` , `EDUCATION` , `MARRIAGE` ) into 29 output features, and lives **inside** each model's `sklearn.pipeline.Pipeline` — so train/test leakage is structurally impossible, and the Streamlit app applies identical preprocessing to whatever a user uploads.

## c. GitHub repository

<https://github.com/shashisingh0015/credit-card-classification>

```

app.py                # Streamlit app – upload, model dropdown, metrics, confusion matrix
requirements.txt       # runtime dependencies (pinned scikit-learn)
README.md             # this file
test_data.csv         # held-out test set (6,000 rows, ~530 KB)
model/
  config.py           # column groups, paths, constants – single source of truth
  data_prep.py        # UCI fetch, cleaning, stratified split
  preprocess.py       # the shared ColumnTransformer every model wraps
  models.py           # one Pipeline factory per classifier + the registry
  train.py            # fits and persists all six models
  evaluate.py         # the 6x6 comparison table (this README's section d)
  viz.py              # shared chart palette (EDA notebook + app)
  eda.ipynb           # exploratory data analysis, 6 figures
  artifacts/          # committed, fitted *.joblib pipelines
reports/
  comparison_table.md # section d's table, regenerated by evaluate.py
  comparison_table.csv
  figures/            # 6 EDA figures referenced in section b
tests/                # 17 contract tests (pytest)
.github/workflows/ci.yml # retrains + re-checks the table on every push

```

Run locally:

```

python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt -r requirements-dev.txt
.\.venv\Scripts\python.exe -m model.data_prep # regenerates data/train.csv
.\.venv\Scripts\python.exe -m model.train     # fits all 6 models -> model/artifacts/
.\.venv\Scripts\python.exe -m model.evaluate  # regenerates the table below
.\.venv\Scripts\streamlit.exe run app.py

```

## d. Model comparison table

All six models share one preprocessing pipeline and the same 5,993-row held-out test set, so the differences below are attributable to the algorithms, not to accidental differences in how each one saw the data. Regenerate with `python -m model.evaluate` (writes this table to `reports/comparison_table.md` and `.csv`); the CI workflow re-derives it from a fresh UCI fetch on every push and fails the build if any metric drifts by more than 0.005 from what's below.

*All-negative baseline (predict "no default" always): 77.87% accuracy, 0.0 recall/F1/MCC, 0.5 AUC.*

| Model                  | Accuracy      | Precision     | Recall        | F1            | AUC-ROC       | MCC           |
|------------------------|---------------|---------------|---------------|---------------|---------------|---------------|
| Logistic Regression    | 0.8136        | <b>0.7267</b> | 0.2526        | 0.3749        | 0.7185        | 0.3515        |
| Decision Tree          | 0.8188        | 0.6846        | 0.3356        | 0.4504        | 0.7447        | 0.3893        |
| Naive Bayes (Gaussian) | 0.5750        | 0.3120        | <b>0.7640</b> | 0.4430        | 0.7300        | 0.2377        |
| k-Nearest Neighbours   | 0.8098        | 0.6317        | 0.3363        | 0.4390        | 0.7412        | 0.3614        |
| Random Forest          | <b>0.8193</b> | 0.6714        | 0.3590        | 0.4678        | 0.7725        | <b>0.3972</b> |
| Gradient Boosting      | 0.8186        | 0.6662        | 0.3612        | <b>0.4685</b> | <b>0.7728</b> | 0.3958        |

## e. Observations and overall winner

**Accuracy is a trap on this dataset, and the table proves it directly.** Five of the six models sit inside a 0.95-point accuracy band (0.8098–0.8193), which would suggest they're nearly interchangeable — yet MCC spans 0.3515–0.3972 and AUC spans 0.7185–0.7728 across those same five models, and Naive Bayes scores *below* the all-negative baseline on accuracy (0.5750) while beating Logistic Regression on AUC (0.7300 vs 0.7185). Any ranking of these six models by accuracy alone would be misleading in both directions. **MCC and AUC-ROC are the metrics this project trusts.**

**Logistic Regression** (MCC 0.3515, AUC 0.7185) has the best **precision** of all six models (0.7267) but the second-worst **recall** (0.2526) — it only flags a client as high-risk when very confident, so it's rarely wrong when it does, but it misses three-quarters of real defaulters. The mechanism is structural: `PAY_0`'s risk is *not* monotonic in its code (section b), and a linear decision boundary cannot represent that inversion. This was predicted from the EDA before any model was fitted, and confirmed here — it's the only model that trails a Decision Tree on every metric.

**Decision Tree** (MCC 0.3893, AUC 0.7447), constrained to `max_depth=5`, `min_samples_leaf=50`, roots its first split on `PAY_0` — exactly the feature the EDA flagged. An unconstrained version of the same tree was also measured: it hits 99.95% train accuracy and then scores *below* the baseline on test data (MCC 0.2126 vs 0.3893), the clearest single illustration in this project of overfitting and why it matters more than train-set performance.

**Naive Bayes (Gaussian)** (MCC 0.2377, AUC 0.7300) is the weakest classifier here by accuracy and MCC, yet has the best **recall** of all six (0.7640) and an AUC that beats Logistic Regression's. Its independence assumption is violated hard by this data — `BILL_AMT1` .6 correlate at 0.80–0.95 — which double- and triple-counts the same evidence, pushes predicted probabilities toward 0 and 1, and wrecks calibration (worst precision, 0.3120) while mostly preserving score *ordering* (respectable AUC). A useful reminder that a "bad classifier" by accuracy can still be a decent *ranker*, and that the fix for a high-AUC/low-precision model is often to move the decision threshold, not to discard the model.

**k-Nearest Neighbours** (MCC 0.3614, AUC 0.7412, k=25, distance-weighted) lands between the linear and tree-based models: it beats Logistic Regression (a local vote can express `PAY_0`'s non-monotonic risk that one coefficient

cannot) but trails the Decision Tree (which finds the  $\text{PAY}_0 \geq 1$  threshold directly, instead of approximating it through distances across 29 scaled features). Its cost profile is the most extreme in the table: the fastest fit of all six (0.03s — fitting a kNN model is just storing the array) but by far the largest artifact (2 MB vs Logistic Regression's 2 KB), since the entire training set has to be persisted for prediction time.

**Random Forest** (MCC **0.3972**, AUC 0.7725; 300 trees, depth 12, `max_features="sqrt"` ) has the best MCC and accuracy of the six. Against the single Decision Tree, MCC rises only  $0.3893 \rightarrow 0.3972$  but AUC rises  $0.7447 \rightarrow 0.7725$  — averaging 300 decorrelated trees sharpens the risk *ordering* substantially more than it changes which side of the 0.5 threshold any given client falls on, which is itself evidence that 0.5 may not be the right threshold for a 22%-positive dataset. Bagging here reduces **variance**, not bias: each tree is deliberately allowed to overfit, and averaging cancels the noise while preserving the signal the trees agree on.

**Gradient Boosting** (MCC 0.3958, AUC **0.7728**; 200 shallow trees, `learning_rate=0.05` ) is a statistical **dead heat** with Random Forest — MCC differs by 0.0014 and AUC by 0.0003, both well inside noise on a 5,993-row test set. It reaches essentially the same ceiling by the opposite mechanism: 200 weak, sequential trees each correcting the previous ensemble's residuals (reducing **bias**), rather than 300 independent trees averaged (reducing **variance**). It took 18.39s to fit versus the forest's 1.35s — boosting's trees are inherently sequential and can't parallelize the way bagging's can.

### Overall winner

**Random Forest**, by the narrowest possible margin over Gradient Boosting, and only on MCC — Gradient Boosting actually wins AUC. Given that near-tie, Random Forest is also the more practical choice operationally: 14× faster to fit, trivially parallelizable, and far less sensitive to its `n_estimators` hyperparameter (more trees costs a forest nothing; it can overfit a boosted model). The honest conclusion this dataset supports is that **both ensembles reach the same performance ceiling** — roughly MCC 0.40 / AUC 0.77 — and that ceiling reflects the signal available in 23 features, not a gap between bagging and boosting as strategies.